

SQL-Checker: Error Detection and Labeling for Text-to-SQL with Interpretability Analysis

Xingyu Ma*
xingyuma@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Xin Tian*
tianxin1860@gmail.com
Wuhan AI Research
Wuhan, China

Lingxiang Wu
lingxiang.wu@nlpr.ia.ac.cn
Institute of Automation, Chinese
Academy of Sciences
Beijing, China

Xuepeng Wang†
xuepeng.wang@ia.ac.cn
Institute of Automation, Chinese
Academy of Sciences
Beijing, China

Xueming Tang‡
xmtang@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Jinqiao Wang§¶
jqwang@nlpr.ia.ac.cn
Institute of Automation, Chinese
Academy of Sciences
Beijing, China

Abstract

Text-to-SQL technology converts natural language queries into SQL statements for database retrieval. Recent advances in large language models (LLMs) have improved Text-to-SQL performance, but generated SQL often contains semantic or syntax errors that degrade user experience and system stability. Existing SQL error detection methods are costly, lack interpretability, and do not support error labeling. To overcome these issues, we propose SQL-CHECKER, a specialized model for Text-to-SQL error detection. We first analyze common error factors in Text-to-SQL, and we design a novel data synthesis framework based on these error factors. This framework simulates error factors to construct a basic error SQL data, and then using an error analysis template to distill high-quality SQL error analysis data from large-scale models. For complex errors, a self-guided iterative distillation strategy further enhances data quality. SQL-CHECKER is then trained on this distilled dataset. Additionally, we refine SQL error labeling and, integrate error label recognition into the detection task, enabling macro-level cause analysis. Experiments show SQL-CHECKER achieves state-of-the-art results on multiple error detection datasets. Incorporating SQL-CHECKER into the Text-to-SQL pipeline also improves execution accuracy.

CCS Concepts

• Information systems → Information retrieval query processing.

Keywords

Text-to-SQL; SQL Error Detection; Large Language Models

*Both authors contributed equally to this research.

†Co-corresponding author.

‡Also with School of Artificial Intelligence, University of Chinese Academy of Sciences.

§Also with Wuhan AI Research.

¶Also with Peng Cheng Laboratory.

ACM Reference Format:

Xingyu Ma, Xin Tian, Lingxiang Wu, Xuepeng Wang, Xueming Tang, and Jinqiao Wang. 2026. SQL-Checker: Error Detection and Labeling for Text-to-SQL with Interpretability Analysis. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792212>

1 Introduction

The Text-to-SQL task aims to convert user natural language into SQL statements, enabling users without a deep understanding of SQL to query databases using natural language. In recent years, significant progress has been made in this field with the application of large language model technologies to the Text-to-SQL task. However, even powerful models like GPT-4 [1] can still produce errors when generating SQL statements. These erroneous SQL statements typically manifest in two ways: **Semantic Errors**, where the query results do not meet the user's expectations; **Syntax Errors**, where the SQL statement fails to execute. The presence of these errors often impacts the user's interaction experience and can even lead to substantial resource consumption, affecting system stability.

Table: employee	Table: Paper	Table: Citation																											
<table><tr><th>emp_id</th><th>...</th><th>lname</th><th>job_lvl</th></tr><tr><td>1</td><td>...</td><td>Cruz</td><td>87</td></tr><tr><td>2</td><td>...</td><td>Roulet</td><td>152</td></tr></table>	emp_id	...	lname	job_lvl	1	...	Cruz	87	2	...	Roulet	152	<table><tr><th>paper_id</th><th>title</th><th>...</th><th>...</th><th>cited_paper_id</th></tr><tr><td>1001</td><td>BusTU...</td><td>...</td><td>...</td><td>A00-1002</td></tr><tr><td>1002</td><td>Machin...</td><td>...</td><td>...</td><td>A00-1002</td></tr></table>	paper_id	title	...	...	cited_paper_id	1001	BusTU...	...	...	A00-1002	1002	Machin...	...	...	A00-1002	
emp_id	...	lname	job_lvl																										
1	...	Cruz	87																										
2	...	Roulet	152																										
paper_id	title	...	...	cited_paper_id																									
1001	BusTU...	...	...	A00-1002																									
1002	Machin...	...	...	A00-1002																									
User's Question: Which job level is O'Rourke at? job level refers to job_lvl	User's Question: Find the id of the papers whose title has the key word 'translation'																												
Generated SQL: SELECT job_lvl FROM employee WHERE lname = 'O'Rourke'	Generated SQL: SELECT t1.paper_id, t1.title FROM paper as t1 JOIN citation as t2 ON t1.paper_id != t2.cited_paper_id																												
Ground Truth: SELECT job_lvl FROM employee WHERE lname = 'O'Rourke'	Ground Truth: SELECT paper_id FROM paper WHERE title LIKE "%translation%"																												
Case 1: Syntax Error	Case 2: Semantic Error																												

Figure 1: Examples of syntax error and semantic error.

As shown in Figure 1. **Case 1:** Because O'Rourke contains an apostrophe, the model generated lname='O'Rourke without properly quoting Rourke, leading to a syntax error. **Case 2:** The generated SQL used unnecessary table joins and the "!=" operator for



joining, causing a Cartesian product. With 26,112 records in the paper table and 124,842 in the citation table, this resulted in over three billion rows, surpassing system capacity and causing crashes.

Existing SQL error detection methods based on large-scale models face challenges such as high cost and poor interpretability. Moreover, current methods generally do not support error labeling, making it hard to analyze error causes at a macro level. We classify these methods into five types: **1) Voting:** Methods like [11, 25, 28] vote on query results to select the most likely correct SQL. However, they cannot detect semantic errors, lack interpretability, and generate high computational cost due to multiple candidate queries. **2) Executor-Based:** Approaches such as [15, 42] treat execution failures as errors. They detect only syntax errors and not semantic ones. **3) Prompting:** Techniques in [5, 8, 41] use LLM prompts combining SQL and user questions to judge correctness. However, closed-source model calls are costly, and prompt-only methods struggle to capture fine-grained semantic errors in complex SQL. **4) Executor & Prompting:** The work [13] fuses execution signals and LLM judgments. It relies on complex rule design, abnormal result signals, and multiple LLM calls, causing high cost. **5) Fine-tune:** The study [7] fine-tunes classifiers for error detection but uses narrow error scenarios and insufficient training data.

Considering the aforementioned issues, it is necessary to investigate an error detection method that can operate on small models (*in this paper, small models denote models with fewer than 14 billion parameters*). Our preliminary approach is to construct erroneous SQL statements to distill the error analysis capability from large models into small models, thereby enabling error detection. Furthermore, we aim to detect error categories to facilitate error statistics analysis at a macro level. This presents the following challenges:

C1: How to construct diverse erroneous SQL statements for distilling analysis data? To enhance the model’s generalization in error detection, it is essential to create SQL statements containing a wide range of error types. Existing methods to generate erroneous SQL can be categorized into two groups: **(1)** collecting erroneous SQL produced by mainstream Text-to-SQL methods [13]. Since these methods perform well on simple and medium difficulty tasks, the resulting error samples are limited and lack diversity; **(2)** generating erroneous SQL using weak or insufficiently trained models [7, 47]. This approach considers only single error sources, leading to limited coverage of error types. To maximize error type coverage, we propose an error factor simulation method for SQL construction. Specifically, we systematically analyze common error factors in Text-to-SQL tasks and simulate them to guide the model to produce diverse mistakes, thereby effectively collecting erroneous SQL statements across various difficulty levels. Simulating multiple error factors significantly improves the diversity of error types.

C2: How to improve the adaptation of small models to distilled data? Our experiments show that error analysis data distilled from large models using erroneous SQL is diverse. However, small models have difficulty performing the full reasoning steps needed for complex tasks [21], which limits their ability to learn effectively from the distilled data. This results in lower detection performance, consistent with [49]. To solve this, we propose a keyword-guided error analysis template. Inspired by task decomposition methods [44], we use SQL keywords as guides to break down the error analysis

task. By applying a templated process, we standardize the analysis and reduce variability, improving small models’ detection ability.

C3: How to distill error analysis data for complex samples? We find that even strong models like GPT-4 often struggle to correctly analyze errors in complex erroneous SQL, making it hard to get accurate analysis for these cases. To address this, we propose a self-guided error analysis generation method. First, we use the erroneous SQL data to produce a batch of analysis results, keeping samples with correct analyses as seed data. For samples with wrong analyses, we reanalyze them using few-shot examples taken from the seed data related to the same question and database. This process repeats until a set maximum number of iterations, gradually improving the analysis accuracy.

C4: How to construct SQL error labels to assist automated statistical analysis? Since error detection usually gives only binary labels showing correctness, it cannot explain the causes of errors. Therefore, more detailed error classification is needed. Many studies have classified SQL errors [26, 27, 30, 40], but their categories are often too fine-grained (e.g., errors in specific clauses). These detailed categories appear during error analysis but are not suitable for macro-level statistical analysis of model performance, since each SQL query has unique fine-grained errors. On the other hand, using only broad categories like syntax or semantic errors does not provide enough useful information. To solve this, and inspired by the two stages of human SQL writing, we define four new error categories.

Table 1: Compared to Other Methods.

Category	Methods	Semantic Check	Analysis Generate	Syntax Check	Error Labeling	Flexible
Voting	[11, 25, 28]	✗	✗	✓	✗	✗
Executor-based	[13, 15, 42]	✗	✗	✓	✗	✓
Prompting	[5, 8, 13, 41]	✓	✓	✗	✗	✓
Finetune	[7]	✓	✗	✗	✗	✓
Finetune	Ours	✓	✓	✓	✓	✓

In summary, Table 1 compares SQL-CHECKER with other SQL error detection methods. SQL-CHECKER supports syntax and semantic detection, error analysis, and error labeling, and can be easily integrated into various Text-to-SQL pipelines. **Our main contributions are summarized as follows:**

- We systematically analyze common error factors in Text-to-SQL and propose a novel erroneous SQL analysis data synthesis framework. This framework simulates error factors to generate basic SQL data, applies keyword-guided error analysis templates for distillation to fit smaller models, and uses a self-guided error analysis generation method to enhance data quality for complex errors.
- We introduce a novel SQL error categorization method. For the first time in SQL error detection, error category recognition is incorporated to facilitate macro-level statistical analysis of error causes.
- We create a SQL error detection dataset with error analyses and labels, and train the SQL-CHECKER model on it using Qwen2.5-Coder-14B-Instruct [17]. Our model achieves state-of-the-art results on multiple error detection benchmarks, outperforming Qwen 2.5-72B-Instruct [46], which was used for data distillation.

2 Overview

This paper is organized into three main parts:

(1) **Error Factors Simulation for SQL Construction:** This part analyze factors causing errors in Text-to-SQL, simulate these factors to guide SQL generation (figure 4(1)), and compare execution results with ground truth to obtain correctness labels (YES or NO). This builds the basic dataset as tuples $(\mathcal{D}, Q, q, C)$, where $\mathcal{D}$ is the database, Q the user query, q the generated SQL, and C the correctness label.

(2) **Error Analysis Generation:** This part proposes a keywords-guided error analysis template to reduce the diversity of analyses and better accommodate the capabilities of smaller models. Next, we design a self-guided analysis generation method (Figure 4(3)) that uses this template to guide the model in generating seed data and iteratively generates new error analyses based on the seed data.

(3) **Error Label Design and Annotation:** This part analyze existing erroneous SQL to create a new error label taxonomy (table 4). Using this taxonomy, error labels are annotated through a hybrid approach combining rule-based methods and LLMs.

3 Error Factors Simulation for SQL Construction

We first analyze factors that may lead to errors in the Text-to-SQL task, hereafter referred to as error factors causing incorrect SQL generation. Next, we simulate these error factors to induce erroneous SQL statements from the LLMs.

3.1 Error Factors Analysis

Our observations indicate that the model is prone to producing incorrect SQL queries under the following four scenarios.

Table 2: The comparison of metrics with(W/) and without(W/O) Scholar training data is presented

Training Set	Scholar-dev%	
	EX	Table/Column Error Rate
W/O Scholar	76.1	86
W/ Scholar	94.3 (↑18.2)	52.4 (↓33.6)

Out-of-Domain(OOD): OOD means the model was not trained on data from the current database domain, causing errors in SQL generation because it cannot properly understand the database. As shown in Table 2, without training on Scholar domain data, the model's EX is 76.1%, and the error rate for column and table names is 86%. After adding 200 training samples from Scholar, EX rises to 94.3%, and the error rate drops to 52.4%. This shows that more domain-specific data helps the model better learn the database schema, improving its ability to extract column and table names and generate accurate SQL.

Out-of-Hardness(OOH): Spider [48] divides SQL queries into four hardness levels: easy, medium, hard, extra hard, based on the number of SQL components, selections, and conditions. For example, a query is hard if it has more than two SELECT columns, more than two WHERE conditions, GROUP BY on two columns, or uses EXCEPT.

We trained the model using datasets of different hardness levels and evaluated it on the SPIDER-DEV set. The model performs well only on the hardness level it was trained on, but poorly on others. As shown in Table 3, after training only on hard queries, the model performs badly on easy ones. We also analyzed the hardness of generated SQL and found it usually matches the training hardness. For example, training on medium hardness leads to medium-hardness SQL outputs, as shown in Figure 7.

Table 3: Comparison of the CodeLlama-7b-Instruct model using training data of varying hardness from the Spider.

Training Set	Spider-dev(EX%)			
	easy	medium	hard	extra hard
<i>spider</i> _{+easy}	88.7	31.2	19.5	7.2
<i>spider</i> _{+medium}	85.5	78.0	21.3	8.4
<i>spider</i> _{+hard}	79.4	60.8	59.8	18.1
<i>spider</i> _{+extra hard}	68.5	58.7	37.9	48.2

Out-of-Knowledge(OOK): OOK refers to when the model is trained on data from the current database domain but still produces incorrect SQL due to missing specific business knowledge. As shown in Figure 2, the model does not know that values in the "Flavor" field of the "goods" table must start with a capital letter. Thus, it generates the condition flavor="apple" instead of the correct flavor="Apple".

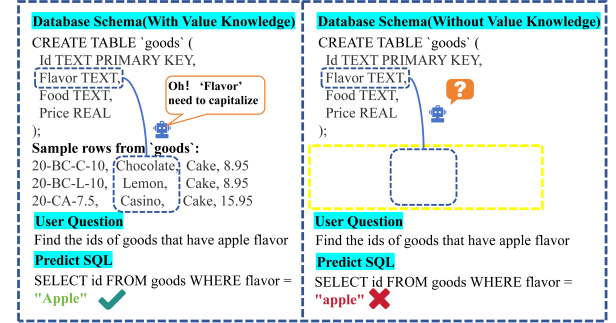


Figure 2: Examples of With Value and Without Value

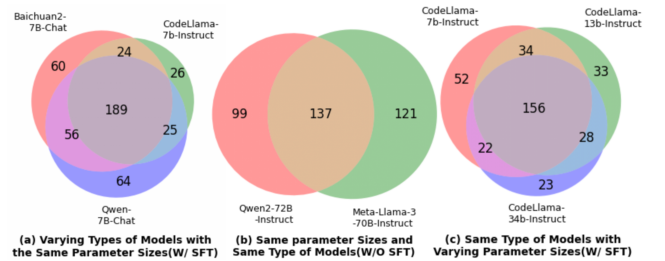


Figure 3: Statistical analysis of erroneous queries across models shows about one-third of errors are unique to each model.

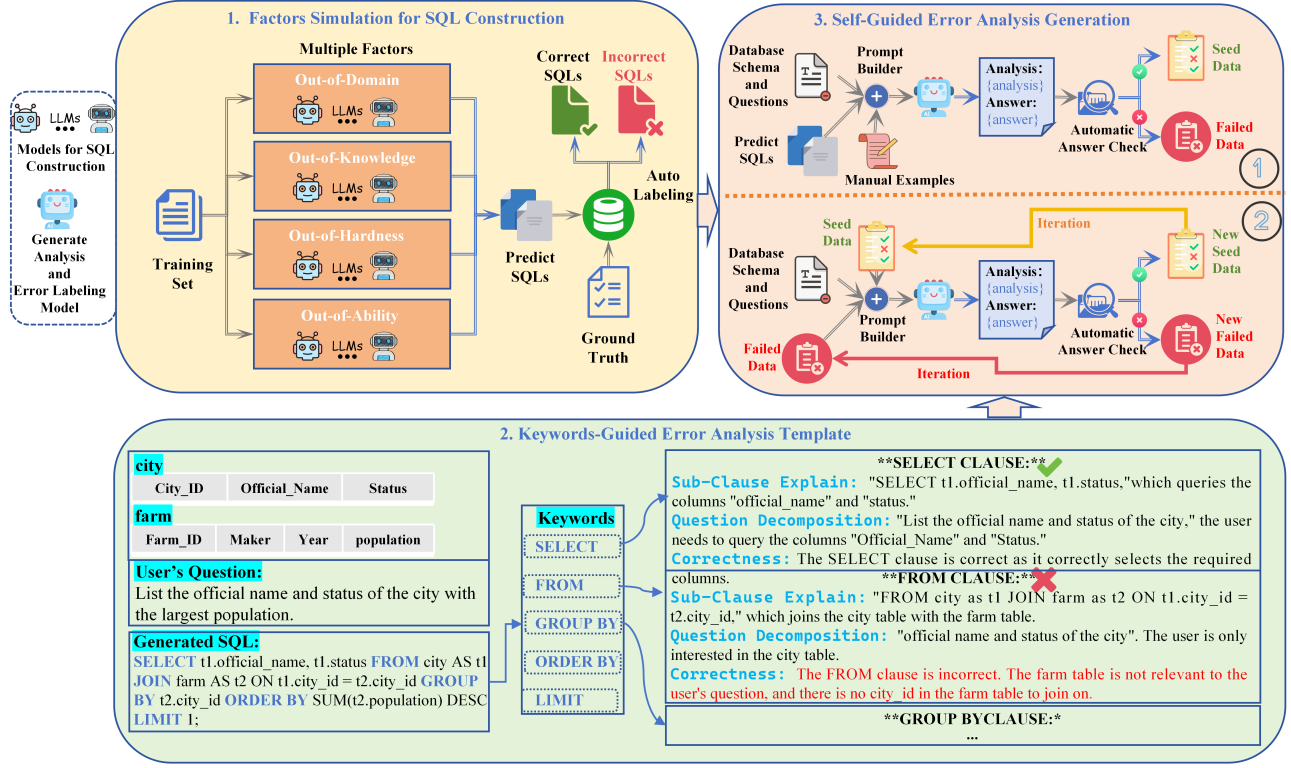


Figure 4: This diagram provides an overview of the work. 1) the top left corner shows the construction of basic data through error factors simulations, 2) The bottom presents the design of the error analysis template, and 3) the top right corner illustrates the self-guided error analysis generation.

Out-of-Ability(OOA): According to the Scaling Law [20], model size affects capability, and there are differences in SQL generation between general and code models. OOA refers to errors in SQL caused by the model's own capabilities. As shown in Figure 3, we ran three sets of experiments with the same training and inference settings. In each experiment, overlapping user questions accounted for only about one-third of the total questions, indicating that each model has its own unique subset of questions on which it tends to make errors. This suggests that each model has characteristic weaknesses, implying that whether a model generates correct or incorrect SQL for the same question could be influenced by its inherent capability factors.

3.2 SQL Construction

As shown in the top left of Figure 4, we simulate the four error factors from Section 3.1 to generate SQL statements. If their execution results match the ground truth, they are labeled correct; otherwise, incorrect. This creates a dataset with correctness labels. Importantly, even when we amplify these simulated errors, the generated SQL still maintains strong semantic relevance to user queries (see A.2.3), showing the errors do not make the SQL unrealistic.

1) **Out-of-Domain:** We group Text-to-SQL data by database domains. Using k-fold cross-validation, we train on one domain

and test on others. Generated SQL is compared to ground truth via execution results to get correct and incorrect samples.

2) **Out-of-Hardness:** Based on Spider's four difficulty levels (easy, medium, hard, extra hard), we train on one level and test on the other three. SQL results are compared to ground truth to label correctness.

3) **Out-of-Knowledge:** We design two prompts for Text-to-SQL: one with only schema information (table and column names) to simulate missing domain knowledge, and one adding sampled database values to simulate available domain knowledge. SQL is generated using LLMs via supervised fine-tuning and prompting.

4) **Out-of-Ability:** Using the above methods on models of different sizes and domains, we take the set difference of erroneous SQL generated by models under the same settings to simulate errors from model capability gaps. For example, in the same OOD setting, SQL generated by CodeLlama-7b-Instruct and Qwen-7B-Chat are compared; errors unique to each model represent capability-related mistakes. The models used are listed in Table 14.

4 Error Analysis Generation

4.1 Keywords-Guided Error Analysis Template Design

As shown in the bottom of Figure 4, our error detection method is divided into five steps:

Table 4: Definition of Error Labels.

Syntax Error	Error Label: General Syntax Error Definition: Indicates there are errors in the order of the SQL statement, missing symbols, or spelling mistakes. Feature: Such SQL cannot be executed in any database. Example: SELECT job_lvl FROM employee WHERE lname= 'O'Rourke Explanation: The single quote is not escaped, and the entire string should be enclosed in double quotes 'O"Rourke'.	Error Label: Database-Specific Syntax Error Definition: The SQL adheres to general syntax rules but cannot execute due to syntax differences across databases. Feature: SQL might run in a specific database but cause syntax errors in others. Example: The CONCAT() function is in MySQL, but it would cause an error in SQLite, since the operator represents string concatenation in SQLite.
	Error Label: Semantic Understanding Error Definition: The model fails to correctly understand the semantics of the user's question, resulting in an SQL statement that has semantic discrepancies with the user's question. Feature: There is a significant semantic difference when converting the SQL statement back into natural language compared to the user's question(SQL→NL). Example:	Error Label: Semantic Generation Error Definition: The model has a basic understanding of the user's question semantics but makes an error when transferring the understood semantics into the corresponding SQL statement. Feature: The SQL transformed into natural language has a similar semantic meaning to the user's question. Example:
Semantic Error	User Question: Show the name and the release year of the song by the youngest singer. Prediction: SELECT Name, Song_release_year FROM singer ORDER BY age LIMIT 1; Ground Truth: SELECT song_name, Song_release_year FROM singer ORDER BY age LIMIT 1; Explanation: The Name in the singer table refers to the singer's name, not the song's name. SQL→NL: "Fetch the name and song release year of the youngest singer", which doesn't match the user's question.	User Question: What are the names of the heads who are born outside the California state? Prediction: SELECT name FROM user WHERE born_state='CA' Ground Truth: SELECT name FROM user WHERE born_state = 'California' Explanation: The generated SQL statement uses the abbreviation 'CA', while the value stored in the database is 'California'. Although both express the same semantic meaning, the mismatch in query conditions results in an empty query result, failing to meet the user's request.

(1) **SQL Keyword Extraction:** Extract SQL keywords from the SQL statement to be detected, denoted as $\mathbb{K} = [K_1^*, K_2^*, \dots, K_n^*]$;

(2) **SQL Clause Splitting:** Use a LLMs to structurally split the SQL statement to be detected based on the SQL keywords $\mathbb{K}$, resulting in a set of all SQL clauses, denoted as $\mathbb{S} = [S_1^*, S_2^*, \dots, S_n^*]$; To avoid excessive segmentation, complex nested queries such as CTE expressions are treated as a whole and not further split.

(3) **Clause Explanation Generation:** Use the LLMs to generate natural language explanations for the SQL clauses $\mathbb{S}$, represented as $E_n^* = f(S_n^*)$;

(4) **Sub-question Decomposition:** Guide the large model with the SQL keywords $\mathbb{K}$ to decompose relevant sub-questions from the user's question, denoted as $\mathbb{Q} = [Q_1^*, Q_2^*, \dots, Q_n^*]$;

(5) **Correctness Analysis:** Finally, use the LLMs to perform a semantic comparison between the natural language explanations of the SQL clauses and the sub-questions to determine correctness, represented as $\mathbb{I}_{[0,1]}(E_n^*, Q_n^*)$.

4.2 Self-Guided Error Analysis Generation

As illustrated in the top right corner of Figure 4, we apply the Keywords-Guided Analysis Template to the SQL dataset constructed in Section 3.2, conducting inference and generation in two stages.

Stage 1: Seed Data Generation. We use the analysis template to construct initial examples that guide the large model in performing

error analysis as designed. The analysis results are then compared with correctness labels. Data with correct analysis results are used as seed data to build the example repository, while data with incorrect analysis results are treated as failure cases for the next iteration. **Stage 2: Self-Guided Refinement.** For the failure cases, we select examples from the seed data to construct few-shot prompts to guide the model in generating error analyses again. Priority is given to examples with the same query and the same database; When the number of such examples is below a threshold, we supplement with examples whose questions have the highest semantic similarity. The inference results are again compared with correctness labels, iteratively expanding the seed data until the maximum number of iterations is reached.

5 Design and Annotation of Error Labels

5.1 Error Label Design

Existing error classification systems are overly fine-grained, making automated analysis difficult. Therefore, we define a new set of error categories (as shown in Table 4). Our study of erroneous SQL shows that syntax and semantic errors can each be divided into two types: **Syntax errors:** (1) SQL does not follow general SQL syntax and cannot run on any database; (2) SQL follows general syntax and runs on some databases but fails on others. **Semantic errors:** (1) When converting SQL back to natural language, it differs significantly

from the original query, indicating that the model likely did not understand the user’s intent; (2) When converting SQL back to natural language, it is semantically similar to the original query, meaning the intent was understood but not correctly expressed in SQL. These two semantic error types correspond to LLMs’ core tasks: natural language understanding and generation.

Based on this, we define four error categories covering different stages, as shown in Table 4, enabling macro-level error analysis. Details in 6.2.6.

5.2 Error Label Annotation

We annotate errors in SQL using rule-based methods and LLMs as follows: (1) **Syntax errors**: We use `sqlglot`¹ to parse SQL AST; if parsing fails, the SQL is marked as a general syntax error. We also collect SQLite-specific functions (e.g., ‘||’) and add them in prompts to help LLM annotate Database-Specific Syntax Errors. (2) **Semantic errors**: Following OmniSQL [22], we convert erroneous SQL back to natural language Q^* . Given schema S , an LLM scores similarity between Q^* and original query Q as $score = LLMs(S, Q^*, Q)$, scoring 1–5. If $score \leq 3$, label as semantic understanding error; if $score > 3$, label as semantic generation error.

6 Experiments

6.1 Experimental Setup

6.1.1 Datasets: Text-to-SQL Datasets: We use Spider [48] and BIRD [24] datasets to test the generalization of our error detection methods. Spider contains 8,659 training, 1,034 development, and 2,147 test samples. BIRD is more challenging, with 9,428 training and 1,534 development samples.

Error Detection Datasets (Ours): Following the data construction methods in Chapters 3 and 4, we built error detection datasets for Spider’s training, development, and test splits, named Sed_{train} , Sed_{dev} , and Sed_{test} . Similarly, from BIRD’s training and development sets, we created Bed_{train} and Bed_{dev} . As Spider and BIRD splits are mutually exclusive, these datasets prevent data leakage. For detailed data distribution, see Appendix A.4.

Public Error Detection Datasets: We also evaluate on public datasets: (1) NL2SQL-BUGs [27], a set from BIRD-dev with 2,018 human-annotated samples; (2) SmBoP, generated by the SmBoP model [35] on Spider’s dev set, as in [7]. These share questions and databases with our datasets but differ in SQL queries, helping validate generalization to other model outputs.

6.1.2 Implementation: Our experiment consists of three stages, each using different models, including both code models and general models (see Table 14). Details are provided in Appendix A.3.

6.1.3 Evaluation Metrics: We evaluated two tasks: 1) **Error Detection**: using F1 scores for positive (F1-pos) and negative (F1-neg) samples, plus overall accuracy. 2) **Text-to-SQL**: using EXECUTION MATCH (EX) [48] as the evaluation metric.

6.1.4 Baselines: We tested both standalone error detection models and their integration into Text-to-SQL frameworks. (1) **Error Detection**: Compared open-source and closed-source models (see

Tables 5 and 6). Open-source focus included Deepseek-Coder-V2-Instruct [50] (236B parameters, code abilities close to GPT-4 [1]) and Qwen2.5-72B-Instruct [46]. Methods compared included SQLens [13], Chain-of-Thought prompting [44], SELF-DEBUG [8], SQL-FixAgent [5], and CHESS [41]. Closed-source models tested were GPT4-o [18], Gemini-2.0-Flash [10], and Claude-3.5-Sonnet [2]. (2) **Text-to-SQL**: To assess error detection’s impact on Text-to-SQL, two frameworks with built-in error detection—CHESS [41] and MAC-SQL [42]—were used. Their detection modules were replaced by our models for evaluation.

6.2 Results and Analysis

6.2.1 Error Detection Results. Tables 5 and 6 present error detection results. Table 5 compares open-source models, and Table 6 shows closed-source results. Due to limited resources, closed-source models were only tested on NL2SQL-BUGs. *Bed* and *Sed* are error detection datasets from BIRD and Spider. To study how training data affects generalization, we used *Sed*, *Bed*, and combined *Sed + Bed* (last three rows of Table 5). Key findings are:

SQL-CHECKER achieves SOTA error detection on multiple datasets. SQL-CHECKER trained on *Bed* and *Sed + Bed* consistently outperforms other models on both open-source (Table 5) and closed-source (Table 6) datasets. It even exceeds Qwen2.5-72B-Instruct, the model used for data creation, supporting our data building method (Chapters 3 and 4). Also, SQL-CHECKER trained on *Sed* outperforms GPT4-o and matches Gemini and Claude-3.5 on NL2SQL-BUGs, while staying SOTA on other datasets.

Our constructed error detection datasets are more challenging. For example, the highest score on our BIRD-based dataset is 67.97%, lower than 77.80% on NL2SQL-BUGs. This means our generated erroneous SQL is closer in meaning to user queries, making errors harder to detect. Similar trends hold for Spider-based datasets. This suggests our SQL generation simulates errors without adding semantic noise.

Our datasets show good generalization. As Table 5 shows, SQL-CHECKER(*Bed*) trained on BIRD performs well on Spider data, proving good generalizability. The same applies to SQL-CHECKER(*Sed*). Results on NL2SQL-BUGs and SmBoP also show our model generalizes to SQL from other generation methods.

Models trained on simpler data improve less in generalization than those trained on complex data. Comparing SQL-CHECKER(*Bed*) and SQL-CHECKER(*Sed*), the model trained on *Sed* has weaker generalization. For example, SQL-CHECKER(*Bed*) gets 80.26% on *Sed_{dev}*, just 1.85% below SQL-CHECKER(*Sed*), but SQL-CHECKER(*Sed*) scores 3.75% lower on *Bed_{dev}*. This may be because the Spider dataset is relatively simpler than the Bird dataset. Also, SQL-CHECKER(*Bed+Sed*) slightly improves on the Spider test set versus SQL-CHECKER(*Bed*), but performs similarly on the BIRD test set. This suggests training with complex SQL data helps improve error detection models.

6.2.2 Effective on Text-to-SQL. We integrated SQL-CHECKER(*Bed*) into two Text-to-SQL methods: MAC-SQL and CHESS. CHESS used DeepSeek-V3(631B parameters) as the base model, while MAC-SQL employed Qwen2.5-72B-Instruct and DeepSeek-Coder-V2-Instruct. We replaced their error detection modules with our SQL-CHECKER(*Bed*). Evaluations were done on *Spider_{dev}*, *Spider_{test}*, and *BIRD_{dev}*,

¹<https://github.com/tobymao/sqlglot/tree/main>

Table 5: Error Detection Results on Open-Source Models

Methods	Data Source: BIRD						Data Source: Spider								
	NL2SQL-BUGS			<i>Bed_{dev}</i>			<i>Sed_{dev}</i>			<i>Sed_{test}</i>			SmBoP		
	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc
DeepSeek-Coder-V2-Lite-Instruct (236B)															
CoT[44]	76.53	73.98	75.32	67.99	53.00	61.91	71.32	77.31	74.66	74.96	79.71	77.58	79.35	79.26	79.31
SQLFixAgent[5]	79.71	72.26	76.56	72.76	44.98	63.55	78.26	79.40	78.84	79.95	81.45	80.72	79.73	80.15	79.94
CHESS[41]	76.02	62.88	70.86	72.06	34.08	60.75	76.28	75.23	75.76	78.50	77.36	77.94	72.15	69.70	70.97
SELEF-DEBUG[8]	74.96	53.67	67.49	72.66	28.43	60.43	78.63	73.87	76.49	79.99	74.85	77.77	80.80	76.88	79.02
SQLens[13]	77.08	72.45	74.97	57.55	38.85	49.92	68.67	73.85	71.49	66.77	73.94	70.78	78.20	77.05	77.63
Qwen2.5-72B-Instruct															
CoT[44]	80.07	73.51	77.25	72.98	41.27	62.98	78.23	78.58	78.40	80.24	80.55	80.39	81.16	82.57	81.89
SQLFixAgent[5]	79.97	72.89	76.95	72.27	44.61	63.04	79.22	80.17	79.70	81.66	82.03	81.84	79.90	81.81	80.90
CHESS[41]	75.69	71.67	73.83	69.57	48.61	61.77	72.19	77.43	75.08	74.26	79.31	77.05	75.25	79.96	77.85
SELEF-DEBUG[43]	75.22	58.54	68.97	71.04	33.16	59.59	78.58	78.72	78.64	80.49	80.16	80.32	77.42	80.04	78.09
SQLens[13]	77.99	74.28	76.36	68.87	50.29	61.71	67.84	74.69	71.61	67.78	75.11	71.91	78.88	78.88	78.88
Qwen2.5-Coder-14B-Instruct (Ours)															
SQL-CHECKER(Bed)	79.67	75.55	77.80	74.44	57.12	67.97	78.63	81.65	80.26	81.90	83.74	82.87	82.41	83.20	82.81
SQL-CHECKER(Sed)	78.95	73.65	76.56	72.44	49.02	64.22	81.25	82.97	82.15	83.63	84.87	84.27	82.38	83.22	82.81
SQL-CHECKER(Bed+Sed)	79.53	75.15	77.55	74.60	56.45	67.92	80.72	82.32	81.55	82.41	83.57	83.01	82.54	83.29	82.92

Table 6: Error Detection Results on Closed-Source Models

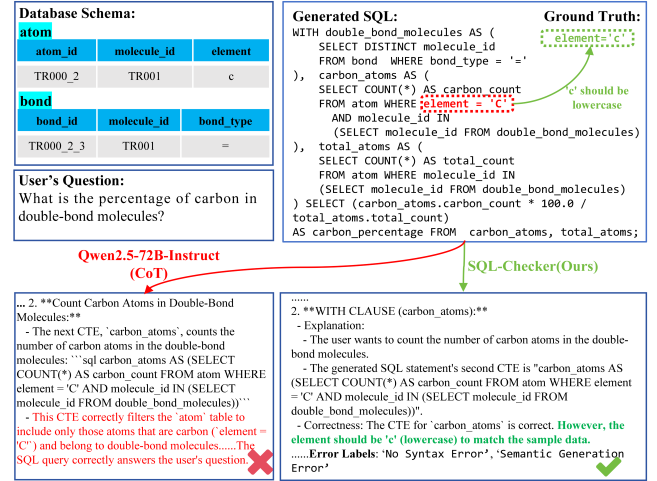
Models	NL2SQL-BUGS		
	F1-pos	F1-neg	ACC
GPT4-o[27]	76.81	78.45	74.89
Gemini-2.0-Flash[27]	78.95	74.17	76.81
Claude-3.5-Sonnet[27]	74.03	78.31	76.36
SQL-CHECKER(Bed)	79.67	75.55	77.80

Table 7: Effective on Text-to-SQL

Methods	Engine Model	SQL-Checker (Bed)	<i>SPIDER_{dev}</i> EX%	<i>SPIDER_{test}</i> EX%	<i>BIRD_{dev}</i> EX%
MAC-SQL [42]	Qwen	W/O	81.8	80.0	54.95
		W/	84.2(↑2.4)	82.1(↑2.1)	57.04(↑2.09)
	DeepSeek	W/O	80.5	79.7	55.80
		W/	82.1(↑1.6)	81.1(↑1.4)	57.17(↑1.37)
CHESS [41]	DeepSeek	W/O	-	-	62.71
		W/	-	-	64.54(↑1.84)

comparing results with (W/) and without (W/O) our SQL-CHECKER. As Table 7 shows, EX improved notably with Qwen2.5-72B-Instruct by 2.4% on *Spider_{dev}*, and 2.1% on both *Spider_{test}* and *BIRD_{dev}*, likely because the error analysis data came from the Qwen model. For DeepSeek-Coder, accuracy gains were 1.6%, 1.4%, and 1.37%, while CHESS improved by 1.84%. These results show that SQL-CHECKER consistently boosts performance across models of different sizes, confirming its effectiveness. Also, we provide visualizations of SQL-CHECKER’s impact on the above pipelines in Appendix A.2.4.

6.2.3 Case study on Error Analysis. As shown in Figure 5, we compare CoT error analysis between Qwen2.5-72B (bottom left) and SQL-CHECKER(Bed) (bottom right). Our model correctly detects an error: the character ‘c’ in `element='C'` should be lowercase. Qwen2.5-72B misses this and wrongly marks the SQL as correct.

**Figure 5: Case study on Error Analysis.**

This proves that our data, generated by simulating errors without external knowledge, helps improve detection performance. Also, our model provides clear error labels like “No Syntax Error” and “Semantic Generation Error” that match our predefined categories. In this case, the SQL semantics are right, but there are subtle implementation errors. Finally, this example demonstrates our model’s ability to analyze errors in complex SQL, such as CTEs, effectively finding fine-grained semantic errors. In contrast, prompt-based methods struggle to do so.

6.2.4 Ablation Study. The ablation results are shown in Table 8. First, we tested the base model’s detection ability using prompting alone (first row). The results show the base model has limited detection capability, proving the need for SFT. Next, we evaluated the self-guided error analysis data generation (second and third rows). Adding the self-guided method improved the F1 score for negative

samples (F1-neg) by 3–5%, showing the two-stage self-guided approach in Section 4.2 effectively boosts detection on hard negatives. Finally, we compared detection models trained on CoT prompting data versus template-based data. As the last two rows show, the model trained with CoT data performed worse, with Bed_{dev} accuracy dropping by 2.09%. This suggests that for smaller models, using a template-based analysis that limits reasoning pattern diversity better matches their reasoning capacity.

Table 8: Ablation Study

Models	Method	NL2SQL-BUGs			Bed_{dev}		
		F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc
Qwen2.5-Coder-14B	Prompt	76.30	59.76	70.16	72.02	32.58	60.45
+template	SFT	79.39	72.56	76.46	74.06	52.88	66.54
+template+sefl-guide		79.67	75.55	77.80	74.44	57.12	67.97
+cot+sefl-guide		79.42	73.83	76.96	72.92	53.91	65.88

6.2.5 Syntax Errors Check. Figure 1 shows that using the database to check SQL syntax errors can cause crashes or failures. So, it is better for the model itself to detect syntax errors. To improve this ability, we included syntax error data and labels when building the SQL-CHECKER training set. The syntax error validation data were taken from samples filtered out during the creation of Bed_{dev} and Sed_{test} , ensuring each question has syntax errors. Specifically, Bed_{dev} contains 1,221 syntax error and 1,469 syntax-correct samples, while Sed_{test} has 1,968 syntax error and 2,139 syntax-correct samples. These subsets, named Bed_{dev} -syntax and Sed_{test} -syntax, were used for syntax error detection experiments.

Table 9: Syntax Error Check Result

Models	Bed_{dev} -syntax			Sed_{test} -syntax		
	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc
DeepSeek-Coder	76.44	58.29	69.88	75.48	74.54	75.02
Qwen2.5-72B-Instruct	75.36	38.33	64.86	78.97	65.60	73.90
SQL-CHECKER(Bed)	85.28	74.91	81.45	91.42	89.49	90.55

As Table 9 shows, even LLMs like Qwen2.5-72B-Instruct and DeepSeek-Coder (236B) have relatively low accuracy detecting syntax errors. This is mainly due to database-specific syntax differences (e.g., SQL dialect syntax errors). After training on syntax error detection, our model greatly improves accuracy on this task.

6.2.6 Error Labels Statistics and Analysis. Chapter 5 introduced new error categories to analyze main error causes in Text-to-SQL tasks. To verify this, we used SQL-CHECKER(Bed) to detect error types in SQL generated by MAC-SQL [42], DIN-SQL [31], and Vanilla [13] on Spider and BIRD datasets. These SQL statements were also pre-annotated using the method from Section 5.2, and statistics were calculated. Figure 6 compares SQL-CHECKER(Bed) detections (blue line) with pre-annotations (red line):

The error label distribution from SQL-CHECKER(Bed) closely matches the pre-annotations, showing **SQL-CHECKER can reliably identify error categories without complex annotation process**. In BIRD, semantic understanding errors are common but slightly less than semantic generation errors. In Spider, semantic

understanding errors are much fewer than semantic generation errors. This reflects real cases since BIRD tasks are more complex, while Spider’s are simpler. **This finding confirms the practical relevance of the proposed semantic error labels.**

Database-specific syntax errors appear more in BIRD but are rare in Spider. This is consistent with the datasets, as BIRD includes more dialect-specific functions (e.g., SQLite’s STRFTIME), while Spider mainly uses common aggregates like MAX and MIN [24]. **This further supports the usefulness of syntax error labels.**

Although SQL-CHECKER(Bed) was trained on BIRD, it also works well detecting errors on Spider, with minor differences in database-specific syntax errors. **This shows good model generalization for error labeling.**

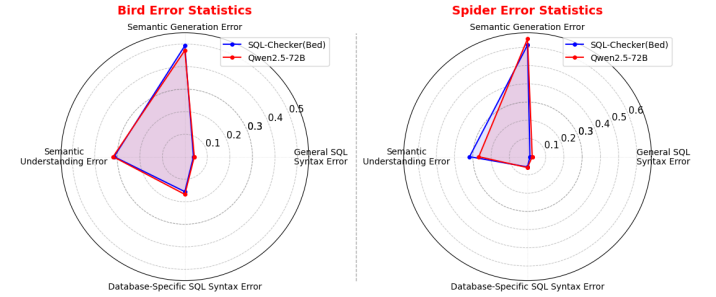


Figure 6: Error Labels Statistics

Statistics show semantic generation errors are more frequent than semantic understanding errors. This suggests that beyond improving comprehension, more effort should focus on enhancing semantic generation, for example, via knowledge injection. For syntax, general syntax errors are mostly resolved; future work should address database-specific syntax errors, for example, by informing LLMs about the SQL execution environment in prompts.

7 Conclusion

This paper proposes an error factor simulation approach for SQL construction and designs an error analysis template tailored for small models. Leveraging this template, we generate an error SQL analysis dataset through a self-guided mechanism. Moreover, we introduce error label recognition to the SQL error detection task for the first time. Based on the constructed error detection dataset, we train the SQL-CHECKER model, which supports both semantic and syntax error detection, provides interpretable error analysis, and incorporates error label recognition. SQL-CHECKER achieves state-of-the-art performance across multiple error detection.

Acknowledgments

This work was supported by National Key R&D Program of China under Grant No.2024YFC3015501, also partly supported by National Natural Science Foundation of China (No. 62276260), the Major Research Plan of Hubei Province under Grant/Award NO. 2023BAA027, the project of Science, Technology and Innovation Commission of Shenzhen Municipality of China under Grant No. GJHZ20240218114659027 and the Key Research & Development Plan of Hubei Province of China under Grant No. 2024BAB049.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Anthropic. 2024. Claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [3] Arian Askari, Christian Poelitz, and Xinye Tang. 2025. Magic: Generating self-correction guideline for in-context text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23433–23441.
- [4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [5] Jipeng Cen, Jiaxin Liu, Zhixu Li, and Jingjing Wang. 2024. SQLFixAgent: Towards Semantic-Accurate SQL Generation via Multi-Agent Collaboration. *arXiv preprint arXiv:2406.13408* (2024).
- [6] Jikai Chen, Leilei Gan, Ziyu Zhao, Zechuan Wang, Dong Wang, and Chenyi Zhuang. 2025. SQLCritic: Correcting Text-to-SQL Generation via Clause-wise Critic. *arXiv:2503.07996 [cs.AI]* <https://arxiv.org/abs/2503.07996>
- [7] Shijie Chen, Ziru Chen, Huan Sun, and Yu Su. 2023. Error detection for text-to-sql semantic parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 11730–11743.
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).
- [9] Ziru Chen, Shijie Chen, Michael White, Raymond Mooney, Ali Payani, Jayanth Srinivasa, Yu Su, and Huan Sun. 2023. Text-to-SQL error correction with language models of code. *arXiv preprint arXiv:2305.13073* (2023).
- [10] Google Deepmind. 2025. Gemini 2.0 Flash Thinking Mode. <https://ai.google.dev/gemini-api/docs/thinking-mode>
- [11] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Jinshu Lin, Dongfang Lou, et al. 2023. C3: Zero-shot text-to-sql with chatgpt. *arXiv preprint arXiv:2307.07306* (2023).
- [12] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [13] Yue Gong, Chuan Lei, Xiao Qin, Kapil Vaidya, Balakrishnan Narayanaswamy, and Tim Kraska. 2025. SQLens: An End-to-End Framework for Error Detection and Correction in Text-to-SQL. *arXiv preprint arXiv:2506.04494* (2025).
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alec Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [15] Chunxi Guo, Zhiliang Tian, Jintao Tang, Pancheng Wang, Zhihua Wen, Kang Yang, and Ting Wang. 2023. Prompting gpt-3.5 for text-to-sql with desemanticization and skeleton retrieval. In *Pacific Rim International Conference on Artificial Intelligence*. Springer, 262–274.
- [16] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [17] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [18] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [19] Guangxin Jiang and Chao Ma. 2025. Exploring Large Language Models for Text-to-SQL Error Correction with LoRA Fine-Tuning. In *2025 8th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*. IEEE, 2670–2674.
- [20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [21] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406* (2022).
- [22] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, et al. 2025. Omnisql: Synthesizing high-quality text-to-sql data at scale. *arXiv preprint arXiv:2503.02240* (2025).
- [23] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
- [24] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [25] Zhishuai Li, Xiang Wang, Jingjing Zhao, Sun Yang, Guoqing Du, Xiaoru Hu, Bin Zhang, Yuxiao Ye, Ziyue Li, Rui Zhao, et al. 2024. PET-SQL: A Prompt-enhanced Two-stage Text-to-SQL Framework with Cross-consistency. *arXiv preprint arXiv:2403.09732* (2024).
- [26] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of NL2SQL with Large Language Models: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* (2024).
- [27] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. 2025. NL2sql-bugs: A benchmark for detecting semantic errors in NL2SQL translation. *arXiv preprint arXiv:2503.11984* (2025).
- [28] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. 2023. Enhancing few-shot text-to-sql capabilities of large language models: A study on prompt design strategies. *arXiv preprint arXiv:2305.12586* (2023).
- [29] Zheng Ning, Yuan Tian, Zheng Zhang, Tianyi Zhang, and Toby Jia-Jun Li. 2024. Insights into natural language database query errors: From attention misalignment to user handling strategies. *ACM Transactions on Interactive Intelligent Systems* 14, 4 (2024), 1–32.
- [30] Zheng Ning, Zheng Zhang, Tianyi Sun, Yuan Tian, Tianyi Zhang, and Toby Jia-Jun Li. 2023. An empirical study of model errors and user error discovery and repair strategies in natural language database queries. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 633–649.
- [31] Mohammadreza Pourreza and Davood Rafiei. 2024. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2024).
- [32] Mohammadreza Pourreza and Davood Rafiei. 2024. Dts-sql: Decomposed text-to-sql with small large language models. *arXiv preprint arXiv:2402.01117* (2024).
- [33] Tonghui Ren, Yuankai Fan, Zhenying He, Ren Huang, Jiaqi Dai, Can Huang, Yinan Jing, Kai Zhang, Yifan Yang, and X. Sean Wang. 2024. PURPLE: Making a Large Language Model a Better SQL Writer. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 15–28. doi:10.1109/ICDE60146.2024.00009
- [34] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [35] Ohad Rubin and Jonathan Berant. 2020. SmBoP: Semi-autoregressive bottom-up semantic parsing. *arXiv preprint arXiv:2010.12412* (2020).
- [36] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv preprint arXiv:2109.05093* (2021).
- [37] Jiawei Shen, Chengcheng Wan, Ruoyi Qiao, Jiazhen Zou, Hang Xu, Yuchen Shao, Yueling Zhang, Weikai Miao, and Geguang Pu. 2025. A Study of In-Context-Learning-Based Text-to-SQL Errors. *arXiv preprint arXiv:2501.09310* (2025).
- [38] Oleg Somov. 2025. The Generalization and Error Detection in LLM-based Text-to-SQL Systems. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. 1077–1079.
- [39] Oleg Somov and Elena Tutubalina. 2025. Confidence Estimation for Error Detection in Text-to-SQL Systems. *arXiv preprint arXiv:2501.09527* (2025).
- [40] Ruoxi Sun, Sercan Ö Arik, Alex Muzio, Lesly Miculicich, Satya Gundabathula, Pengcheng Yin, Hanjun Dai, Hootan Nakhost, Rajarishi Sinha, Zifeng Wang, et al. 2023. Sql-palm: Improved large language model adaptation for text-to-sql (extended). *arXiv preprint arXiv:2306.00739* (2023).
- [41] Shayan Talaie, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755* (2024).
- [42] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. 2024. Mac-sql: A multi-agent collaborative framework for text-to-sql. *arXiv preprint arXiv:2312.11242* (2024).
- [43] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [45] Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, et al. 2023. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305* (2023).
- [46] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [47] Jiayi Xiang, Binyuan Hui, Min Yang, Jian Yang, Junyang Lin, and Chang Zhou. 2024. Synthesizing Text-to-SQL Data from Weak and Strong LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7864–7875.

- [48] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanell Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887* (2018).
- [49] Zhexin Zhang, Yida Lu, Jingyuan Ma, Di Zhang, Rui Li, Pei Ke, Hao Sun, Lei Sha, Zhifang Sui, Hongning Wang, et al. 2024. Shieldlm: Empowering llms as aligned, customizable and explainable safety detectors. *arXiv preprint arXiv:2402.16444* (2024).
- [50] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *arXiv preprint arXiv:2406.11931* (2024).

A Appendix

A.1 Error Factors Analysis

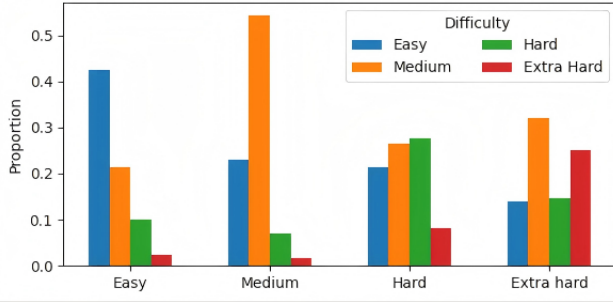


Figure 7: Analysis of SQL generation hardness: the horizontal axis shows training data hardness, and the vertical axis shows the proportion of generated data at each hardness level

A.2 Supplementary Experiments

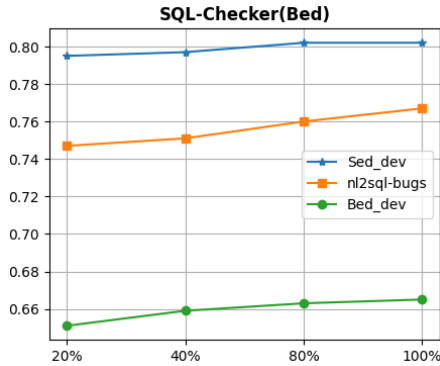


Figure 8: Impact of SQL Number on Error Detection

A.2.1 Diversity Validation of Erroneous SQL Statements. Since we cannot directly count the types of SQL errors, we tested how the number of SQL statements affects error detection to verify the diversity of SQL error types. For each question, we generated multiple SQL statements, but the number of questions stayed the same. If the error types were all the same and only the number of SQL increased, the model would learn only a few error types, so detection

performance would not improve with more SQL. Using this idea, we indirectly checked the diversity of generated SQL. We sampled 20%, 40%, 80%, and 100% of the data and trained for 2 epochs each. As shown in Figure 8, error detection accuracy improves significantly with more SQL statements. This also suggests that the new SQL statements are diverse, not just very similar.

Table 10: Data Generality Validation on Different Models

Models	NL2SQL-BUGs			Bed _{dev}		
	F1-pos	F1-neg	Acc	F1-pos	F1-neg	Acc
Qwen2.5-Coder-7B-Instruct	78.47	70.83	75.22	72.88	55.96	66.43
Meta-Llama-3-8B-Instruct	75.76	71.81	73.93	71.34	57.52	65.77
deepseek-coder-6.7b-instruct	77.81	72.58	75.47	72.08	57.14	66.18

A.2.2 Data Generality Validation. We compared the error detection performance of the dataset on a heterogeneous model [49] to demonstrate the generalizability of the data. The experimental results are presented in Table 10. As shown in the table, the Llama model performs slightly worse than Qwen2.5 and deepseek, which may be attributed to the inherent advantages of coder models. The detection capabilities of Qwen2.5 and deepseek-coder are almost identical, indicating that although the error detection data in this work were generated using the Qwen series models, the generated data are equally applicable to models with different architectures.

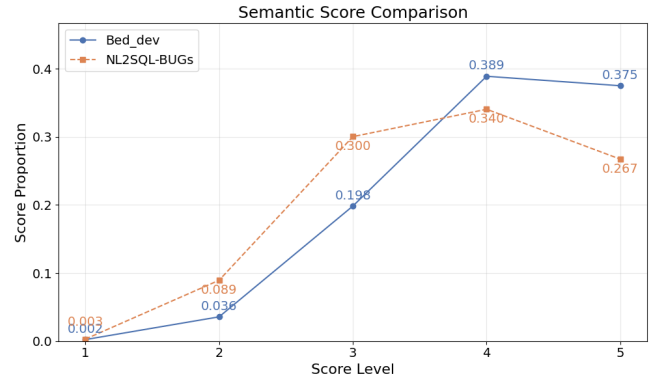


Figure 9: Query Semantic Similarity Comparison

A.2.3 Query Semantic Similarity Comparison: To assess semantic correlation between SQL queries and original user questions, we converted SQL statements from Bed_{dev} and NL2SQL-BUGs into natural language questions. Then, following Section 5.2, an LLM computed semantic relevance scores between reconstructed questions Q^* and original questions Q .

The comparative results for the two datasets are presented in the figure 9. It is evident that the proportion of samples with scores above 3 in Bed_{dev} is significantly higher than that in NL2SQL-BUGs. This difference helps explain why the detection task on our dataset is more challenging. Furthermore, these findings indicate that the SQL statements generated via our error factor simulation method remain consistent with the semantics encountered in real-world scenarios.

A.2.4 Case study on SQL Repair. We further analyze how the SQL-CHECKER affects erroneous SQL correction in Text-to-SQL pipelines (experiment in Section 6.2.2), as shown in Figure 10. **Case 1:** The value "70" in the user's question is a string, but the generated SQL treats it as an INTEGER. The model understood the query semantics but made a semantic generation error when converting it to SQL. Our model correctly detected this error and helped fix it. **Case 2:** The SQL includes unnecessary table joins using "!= ", causing a Cartesian product. With over 3 billion records, this crashed the SQLite service. Our model detected this issue and assisted in resolving it successfully.

## Question: Which good has "70" in its id? And what is its price? ## First Generated: ❌ SELECT 'Flavor', 'Food', 'Price' FROM goods WHERE Id = 70 ## SQL-Checker Analysis ...The SELECT clause and FROM clause of the SQL statement are correct, but the WHERE clause is incorrect. The WHERE clause should use the 'LIKE' operator to check if the 'Id' contains the string "70". ## Re-generated with Analysis: ✅ SELECT 'Flavor', 'Food', 'Price' FROM goods WHERE Id LIKE "%70%"	## Question: Find the id of the papers whose title has the key word 'translation' ## First Generated: ❌ SELECT t1.paper_id, t1.title FROM paper as t1 JOIN citation as t2 ON t1.paper_id != t2.cited_paper_id ## SQL-Checker Analysis ...The FROM clause is incorrect. The 'Citation' table is not relevant to the user's question, and the join condition 't1.paper_id != t2.cited_paper_id' is not appropriate for the user's query... ...The absence of a WHERE clause is incorrect. The query should include a WHERE clause to filter the results based on the title... ## Re-generated with Analysis: ✅ SELECT paper_id FROM paper WHERE title LIKE "%translation%"
---	--

Case 1: Semantic Generation Error

Case 2: Semantic Understanding Error

Figure 10: Case study on SQL repair.

A.2.5 Detection accuracy under different error factors. We have supplemented the evaluation of SQL-CHECKER on error SQL datasets constructed with different error factors.

Table 11: Detection accuracy under different error factors.

Model	ACC(%)			
	OOD	OOK	OOH	OOA
Qwen2.5-72B-Instruct	66.16	55.70	72.61	72.15
SQL-CHECKER	72.32	65.04	76.80	78.35

Result Analysis: OOD, OOK, OOH, and OOA represent the four types of error SQL generated by simulating error factors in Section 3.2. We use OOK as an example because its error detection scores are the lowest. This is expected since the model can only infer limited external knowledge from the prompt. For example, as shown in Figure 2, where `flavor='apple'` and where `flavor='Apple'` are semantically the same, which makes error detection difficult. To address this, we simulated knowledge gaps when creating the error data. As a result, SQL-CHECKER can detect these errors better, reaching 65.04%, which is much higher than Qwen2.5-72B's 55.70%. Also, in Figure 5, we visualize how Qwen2.5-72B and SQL-CHECKER detect these errors. SQL-CHECKER successfully finds case-related errors in conditions, while Qwen2.5-72B does not. These results show that our method of simulating error factors to build error data is effective.

A.2.6 Error Detection Performance of Keywords-Guided Error Analysis without SFT. To verify that our error detection method does not rely on gains from SFT, we compared CoT and the Keywords-Guided Error Analysis method using prompts on Qwen2.5-72B-Instruct. As shown in Table 12, our error detection approach still outperforms CoT using only prompts, demonstrating the effectiveness of our method.

Table 12: Prompting-based error detection results

Methods	Model	NL2SQL-NUGs			Bed_dev		
		F1-pos	F1-neg	ACC	F1-pos	F1-neg	ACC
CoT	Qwen2.5-72B	80.07	73.51	77.25	72.98	41.27	62.98
Ours	-Instruct	80.72	75.30	78.34	72.51	47.64	63.95

A.2.7 Detection Efficiency Evaluation. We evaluated our detection model's efficiency on the 8*A800 80G, as shown in Table 13. While the model is slightly slower than the base model due to more inference tokens, it has significantly higher accuracy (see 6.2.4 Ablation Study). Additionally, it demonstrates clear advantages in both efficiency and accuracy compared to the 72B model

Table 13: Detection Efficiency Evaluation

Models	Average Response Time(s)
Qwen2.5-72B-Instruct	10.19s
Qwen2.5-Coder-14B-Instruct	4.95s
SQL-CHECKER(Ours)	6.41s

A.2.8 Evaluation of Error Analysis. To validate the rationality of error analysis and labeling, we randomly selected 100 distinct questions from the Bed_{dev} dataset. For each, one erroneous SQL statement was randomly chosen and analyzed by the SQL-CHECKER model. We manually evaluated the analysis and labeling based on: **1) Reasonable:** error causes accurately identified and labels correctly assigned; **2) Ambiguous:** inherent data issues (e.g., ambiguous user questions) or partial error cause and label identification; **3) Unreasonable:** error causes and labels do not correspond to the actual errors. Among the 100 questions, 92 analyses were deemed fully reasonable, 3 involved ambiguous data, and only 5 were judged as unreasonable. Furthermore, 94 error labelings were considered reasonable, while only 6 were unreasonable. These results demonstrate that the SQL-CHECKER's analysis process and error labeling are of practical utility.

A.3 Implementaion Details

Stage 1: Factor Simulation for SQL Construction. Stage 1 corresponds to the models employed in Section 3.2 for constructing the basic SQL dataset. During the SQL data construction phase, both SFT-based and prompting-based methods were applied. Specifically, for the SFT-based approach, we employed the open-source repository DB-GPT-HUB². The learning rate was set to 2×10^{-5} , with a cosine annealing scheduler incorporating restarts. The batch

²<https://github.com/eosphoros-ai/DB-GPT-Hub>

Table 14: Model list of our work.

Domain	Stage 1: Factor Simulation for SQL Construction		Stage 2: Analysis Generation and Error Label Annotation	Stage 3: Error Detection Model Training
	SFT-based	Prompting-based		
General Models	Baichuan2-7B-Chat[45] Qwen-7B-Chat[4]	Meta-Llama-3-8B-Instruct[14] Meta-Llama-3-70B-Instruct[14] Qwen2-72B-Instruct Qwen1.5-72B-Chat DeepSeek-Chat-236B[50]	Qwen2.5-72B-Instruct[46]	None
	deepseek-coder-6.7b-instruct[16] deepseek-coder-33b-instruct[16] CodeLlama-7b-Instruct[34] CodeLlama-13b-Instruct[34] CodeLlama-34b-Instruct[34]	DeepSeek-Coder-V2-Lite-Instruct[50] deepseek-coder-33b-instruct[16] Deepseek-coder-236B[50] CodeLlama-34b-Instruct[34] Qwen2.5-Coder-7B-Instruct[17]	None	Qwen2.5-Coder-14B-Instruct[17] Qwen2.5-Coder-7B-Instruct[17] DeepSeek-coder-6.7b-Instruct[16] Meta-Llama-3-8B-Instruct

size was 2, number of epochs 5, max_source_length 2048, and max_target_length 512. For models exceeding 32B parameters, DeepSpeed stage 3 optimization was used; otherwise, stage 2 was applied. For the prompting-based method, models were deployed via lmdeploy with parameters: temperature 0.95, top_p 0.7, and top_k 50.

Stage 2: Analysis Generation and Error Label Annotation. Stage 2 involves the models applied in Sections 4.2 and 5.2, which iteratively generate error analysis data based on the basic SQL dataset and annotate erroneous SQL statements. To balance cost and effectiveness, Qwen2.5-72B-Instruct [46] is adopted. In the error analysis generation phase, the model deployed via lmdeploy³ used a temperature of 1, while other inference parameters followed the OpenAI⁴ library defaults.

Stage 3: Error Detection Model Training. Stage 3 is the training phase for the error detection model. The primary experiments utilize the Qwen model. Additionally, to demonstrate the generalizability of the constructed datasets, heterogeneous models[49] such as Deepseek, and Llama3 were also evaluated. During the detection model training phase, the number of epochs was set to 10, batch size to 1, and both max_source_length and max_target_length to 4096. The learning rate and scheduler remained at 2×10^{-5} and cosine annealing with restarts, respectively. Training was conducted on 4 machines, each equipped with 8 NVIDIA A800-SXM4-80GB GPUs, with a total training time of approximately 72 hours.

Table 15: Data Distribution

Dataset	Positive		Negative		Error Type	
	Question	SQL	Question	SQL	Syntax	Semantic
<i>Sed_{train}</i>	8,193	62,561	8,284	100,932	53,822	47,110
<i>Sed_{dev}</i>	833	2,168	833	2,375	0	2,375
<i>Sed_{test}</i>	1,788	3,391	1,788	3,578	0	3,578
<i>Bed_{train}</i>	6,754	66,331	7,094	132,299	67,697	64,602
<i>Bed_{dev}</i>	949	3,015	949	2,583	0	2,583

³<https://github.com/InternLM/lmdeploy>⁴<https://github.com/openai/openai-python>

A.4 Data Distribution:

Table 15 shows data distribution including counts of positive and negative examples for questions and SQL, noting that a question may link to multiple SQL statements. Both *Sed_{train}* and *Bed_{train}* contain syntax and semantic errors, but test and dev sets focus on semantic errors, with syntax error detection results also reported.

A.5 Related Work

Text-to-SQL: The Text-to-SQL task aims to translate natural language queries into SQL statements. This task can be broadly categorized into two approaches: Training-based methods [23, 32, 36] and Prompting-based methods [11, 12, 31, 42].

Errors in Text-to-SQL: Errors in Text-to-SQL refer to generated SQL queries that either do not align with user intent or contain syntax errors. Many studies (e.g., [26, 27, 29, 31, 37, 41]) analyze and categorize these errors; however, these categorizations are often too fine-grained, sometimes down to the operator level, which limits the feasibility of broad statistical analysis. Moreover, NL2SQL-BUGs [27] provides a benchmark for error detection, while studies such as [3, 6, 9, 19, 33] focus on error correction.

Error Detection for Text-to-SQL: Current SQL error detection techniques fall into these categories: **1) Manual:** Compare execution results with ground truth (e.g., Spider [48], BIRD [24]). **2) Voting:** Generate multiple SQL candidates and select the majority-consistent one [11, 25, 28] using self-consistency [43]; cannot detect semantic errors. **3) Executor-Based:** Execute SQL on the database to catch exceptions and prompt LLMs for corrections [15, 42]; only detects syntax errors. **4) Prompting:** Such as SQLFixAgent [5], SQLCritic[6] and self-debug[8] using LLMs to analyze SQL and user queries to judge correctness; struggle to capture fine-grained semantic errors in complex SQL. **5) Executor and Prompting:** SQLens [13] combines LLM and execution signals but relies on complex design and misclassifies empty results as errors. **6) Fine-tuning:** Models fine-tuned with CodeBERT and GAT [7] detect errors but have limited erroneous sample collection and no error cause analysis. **7) Others:** Confidence estimation methods [38, 39] detect errors but lack interpretability.